

Domaća naloga 7 – Ibrahim Čaušević

KODA:

```
25 </> class MainActivity : ComponentActivity() {  
26  
27     1 Usage  
28     private val apiKljuc = "800046066f87a8e986c1e56fefdc4899"  
29  
30     override fun onCreate(savedInstanceState: Bundle?) {  
31         super.onCreate(savedInstanceState)  
32         setContent {  
33             VremenskaApplikacija(apiKljuc)  
34         }  
35     }  
36  
37     4 Usages  
38     data class VremeInfo(  
39         val mesto: String,  
40         val cas: String,  
41         val opis: String,  
42         val temperatura: String,  
43         val emoji: String,  
44         val veter: String,  
45         val vlaga: String,  
46         val sonciVzhod: String,  
         val sonciZahod: String  
    )
```

```

47 @Composable
48 fun VremenskaAplikacija(kljuc: String) {
49     val mesta = listOf("Ljubljana", "Maribor", "Koper")
50
51     var izbranoMesto by remember { mutableStateOf( value = mesta[0]) }
52     var razsirjeno by remember { mutableStateOf( value = false) }
53     var vreme by remember { mutableStateOf<VremeInfo?>( value = null) }
54     var napaka by remember { mutableStateOf( value = "") }
55
56     LaunchedEffect( key1 = izbranoMesto) {
57         try {
58             vreme = pridobiVreme(izbranoMesto, kljuc)
59             napaka = ""
60         } catch (e: Exception) {
61             napaka = "Nekaj je šlo narobe pri vremenu."
62         }
63     }
64
65     Column(
66         modifier = Modifier
67             .fillMaxSize()
68             .background(Color(0xFF0878A8))
69             .padding( all = 20.dp),
70         horizontalAlignment = Alignment.CenterHorizontally
71     ) {
72         Spacer(modifier = Modifier.height(40.dp))
73
74         Text(
75             text = "Vreme",
76             color = Color.White,
77             fontSize = 30.sp,
78             fontWeight = FontWeight.Bold
79         )
80
81         Spacer(modifier = Modifier.height(20.dp))

```

```

83     Box {
84         Button(onClick = { razsirjeno = true }) {
85             Text(izbranoMesto)
86         }
87
88         DropdownMenu(
89             expanded = razsirjeno,
90             onDismissRequest = { razsirjeno = false }
91         ) {
92             mesta.forEach { m ->
93                 DropdownMenuItem(
94                     text = { Text(m) },
95                     onClick = {
96                         izbranoMesto = m
97                         razsirjeno = false
98                     }
99                 )
100             }
101         }
102     }
103
104     Spacer(modifier = Modifier.height(30.dp))
105
106     if (napaka.isNotEmpty()) {
107         Text(napaka, color = Color.White)
108     } else if (vreme == null) {
109         CircularProgressIndicator(color = Color.White)
110     } else {
111         Vreme(v = vreme!!)
112     }
113 }
114 }

```

```

116 @Composable
117 fun Vreme(v: VremeInfo) {
118     Column(
119         modifier = Modifier.fillMaxWidth(),
120         horizontalAlignment = Alignment.CenterHorizontally
121     ) {
122         Text("${v.mesto}, ${v.cas}", color = Color.White.copy(alpha = 0.8f))
123         Spacer(modifier = Modifier.height(20.dp))
124
125         Text(
126             text = v.opis.replaceFirstChar { it.uppercase() },
127             color = Color.White,
128             fontSize = 28.sp,
129             fontWeight = FontWeight.Bold
130         )
131
132         Spacer(modifier = Modifier.height(20.dp))
133
134         Row(verticalAlignment = Alignment.CenterVertically) {
135             Text(
136                 text = v.temperatura,
137                 color = Color.White,
138                 fontSize = 70.sp,
139                 fontWeight = FontWeight.Bold
140             )
141
142             Spacer(modifier = Modifier.width(25.dp))
143
144             Box(
145                 modifier = Modifier
146                     .size(110.dp)
147                     .background(Color.White.copy(alpha = 0.25f), CircleShape),
148                 contentAlignment = Alignment.Center
149             ) {
150                 Text(v.emoji, fontSize = 55.sp)
151             }
152         }
153
154         Spacer(modifier = Modifier.height(30.dp))

```

```

155
156 InfoVrstica( oznaka = "Hitrost vetra", vrednost = v.veter)
157 InfoVrstica( oznaka = "Vlažnost", vrednost = v.vlaga)
158 InfoVrstica( oznaka = "Sončni vzhod", vrednost = v.sončniVzhod)
159 InfoVrstica( oznaka = "Sončni zahod", vrednost = v.sončniZahod)
160
161 Spacer(modifier = Modifier.height(30.dp))
162
163 Text(
164     text = "Napoved za naprej",
165     color = Color.White,
166     fontSize = 18.sp,
167     fontWeight = FontWeight.Bold
168 )
169
170 Spacer(modifier = Modifier.height(10.dp))
171
172 NapovedVrstica( cas = "Čez 1 uro", temp = v.temperatura, v.emoji)
173 NapovedVrstica( cas = "Čez 2 uri", temp = v.temperatura, v.emoji)
174 NapovedVrstica( cas = "Čez 3 ure", temp = v.temperatura, v.emoji)
175 }
176 }
177
178 4 Usages
179 @Composable
180 fun InfoVrstica(oznaka: String, vrednost: String) {
181     Row(
182         modifier = Modifier
183             .fillMaxWidth()
184             .padding(vertical = 6.dp),
185         horizontalArrangement = Arrangement.SpaceBetween
186     ) {
187         Text(oznaka, color = Color.White.copy(alpha = 0.75f))
188         Text(vrednost, color = Color.White, fontWeight = FontWeight.Bold)
189     }
190 }

```

```

191 @Composable
192 fun NapovedVrstica(cas: String, temp: String, emoji: String) {
193     Row(
194         modifier = Modifier
195             .fillMaxWidth()
196             .background(Color.White.copy(alpha = 0.10f))
197             .padding( all = 12.dp),
198         verticalAlignment = Alignment.CenterVertically,
199         horizontalArrangement = Arrangement.SpaceBetween
200     ) {
201         Text(cas, color = Color.White)
202         Text(temp, color = Color.White, fontSize = 22.sp)
203         Text(emoji, fontSize = 28.sp)
204     }
205
206     Spacer(modifier = Modifier.height(6.dp))
207 }

```

```

209 suspend fun pridobiVreme(mesto: String, kljuc: String): VremeInfo {
210     return withContext(Dispatchers.IO) {
211         val urlTekst = "https://api.openweathermap.org/data/2.5/weather?q=$mesto,SI&appid=$kljuc&units=metric&lang=sl"
212
213         val url = URL( spec = urlTekst)
214         val povezava = url.openConnection() as HttpURLConnection
215         povezava.requestMethod = "GET"
216
217         val odziv = povezava.inputStream.bufferedReader().readText()
218         val json = JSONObject(odziv)
219
220         val vremeObject = json.getJSONArray( name = "weather").getJSONObject( index = 0)
221         val glavno = json.getJSONObject( name = "main")
222         val veterObject = json.getJSONObject( name = "wind")
223         val sonceObject = json.getJSONObject( name = "sys")
224
225         val opis = vremeObject.getString( name = "description")
226         val temp = glavno.getDouble( name = "temp").toInt()
227         val vlaga = glavno.getInt( name = "humidity")
228         val hitrostVetra = veterObject.getDouble( name = "speed")
229         val smerVetra = veterObject.optInt( name = "deg", fallback = 0)
230
231         val vzhod = sonceObject.getLong( name = "sunrise")
232         val zahod = sonceObject.getLong( name = "sunset")
233         val casMeritve = json.getLong( name = "dt")
234
235         VremeInfo(
236             mesto = mesto,
237             cas = formatirajCas( sekunde = casMeritve),
238             opis = opis,
239             temperatura = "$temp °C",
240             emoji = emojiVremena(opis),
241             veter = "$hitrostVetra m/s, ${smerVetraString( stopinje = smerVetra)}",
242             vlaga = "$vlaga %",
243             soncniVzhod = formatirajCas( sekunde = vzhod),
244             soncniZahod = formatirajCas( sekunde = zahod)
245         )
246     }
247 }

```

```

249 fun formatirajCas(sekunde: Long): String {
250     val datum = Date(sekunde * 1000)
251     val format = SimpleDateFormat( pattern = "HH:mm", Locale.getDefault())
252     return format.format( date = datum)
253 }
254
255 1 Usage
256 fun emojiVremena(opis: String): String {
257     val o = opis.lowercase()
258
259     return when {
260         o.contains( other = "dež") || o.contains( other = "rain") -> "☁️ "
261         o.contains( other = "sneg") || o.contains( other = "snow") -> "❄️ "
262         o.contains( other = "megla") || o.contains( other = "fog") || o.contains( other = "mist") -> "🌫️ "
263         o.contains( other = "obla") || o.contains( other = "cloud") -> "☁️ "
264         o.contains( other = "jasno") || o.contains( other = "clear") -> "☀️ "
265         else -> "⚡️ "
266     }
267 }
268 1 Usage
269 fun smerVetraString(stopinje: Int): String {
270     return when (stopinje) {
271         in 0 ≤ .. ≤ 22 -> "S"
272         in 23 ≤ .. ≤ 67 -> "SV"
273         in 68 ≤ .. ≤ 112 -> "V"
274         in 113 ≤ .. ≤ 157 -> "JV"
275         in 158 ≤ .. ≤ 202 -> "J"
276         in 203 ≤ .. ≤ 247 -> "JZ"
277         in 248 ≤ .. ≤ 292 -> "Z"
278         in 293 ≤ .. ≤ 337 -> "SZ"
279         else -> "S"
280     }

```

TEST:



